WEEK 9

Write a program to Visualizing Convolutional Neural Networks(display result of convolution and pooling operations as feature ma

+ Code + Markdown

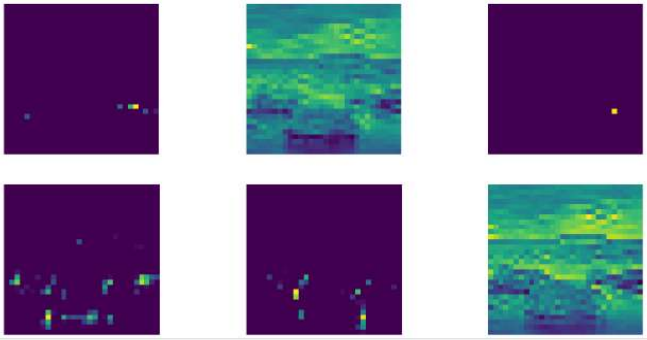
[49]:
```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras import layers, models
```

```
def simple_cnn():
    model = models.Sequential([
        layers.Input(shape=(32,32,3)),

        layers.Conv2D(16, (3,3), activation='relu', name='conv1'),
        layers.MaxPooling2D((2,2), name='pool1'),

        layers.Conv2D(32, (3,3), activation='relu', name='conv2'),
        layers.MaxPooling2D((2,2), name='pool2')
    ])
    return model

model = simple_cnn()

# select image
img = X_test[0]
img = np.expand_dims(img, axis=0)

# IMPORTANT FIX
model.predict(img)   # build the model

# extract layers
layer_outputs = [layer.output for layer in model.layers]

# FIX HERE
activation_model = models.Model(inputs=model.layers[0].input, outputs=layer_outputs)

# get activations
activations = activation_model.predict(img)
# get activations
activations = activation_model.predict(img)
```

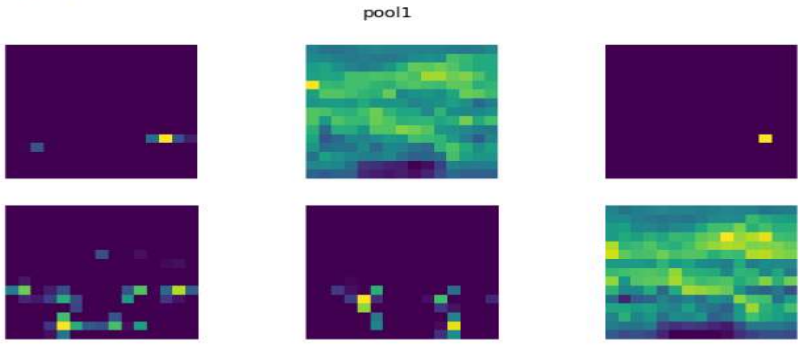
```
def plot_feature_maps(activations, layer_names):
    for layer_name, feature_map in zip(layer_names, activations):
        print(f"\nLayer: {layer_name}")

        num_filters = feature_map.shape[-1]

        plt.figure(figsize=(10,5))

        for i in range(min(num_filters, 6)):  # show first 6 filters
            plt.subplot(2,3,i+1)
            plt.imshow(feature_map[0, :, :, i], cmap='viridis')
            plt.axis('off')

        plt.suptitle(layer_name)
        plt.show()

layer_names = [layer.name for layer in model.layers]
plot_feature_maps(activations, layer_names)
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 278ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 377ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 47ms/step

Layer: conv1
```

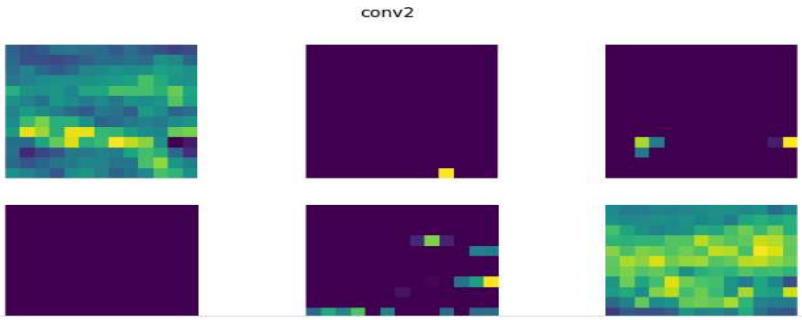
conv1



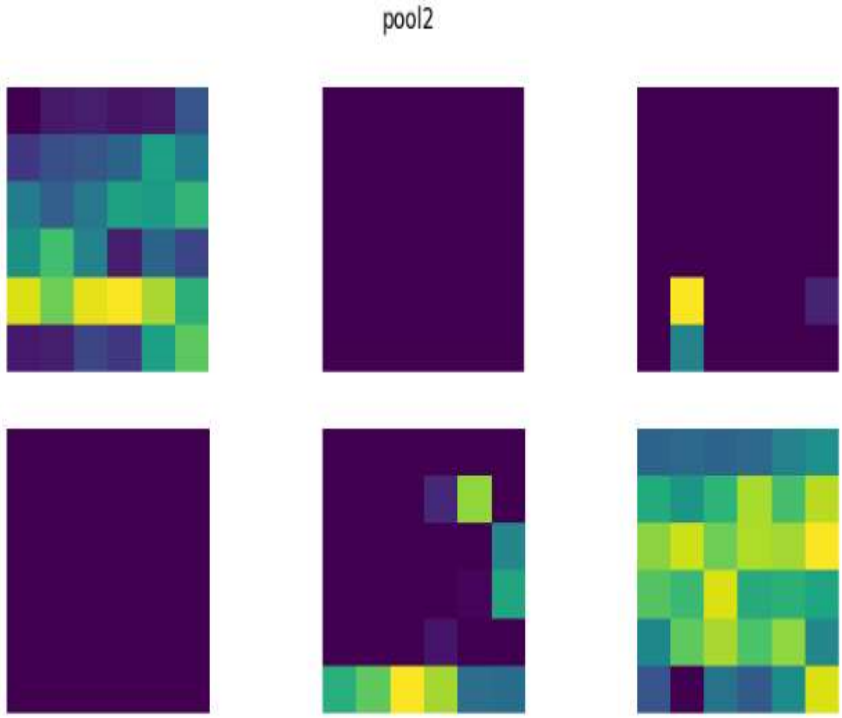
Layer: pool1



Layer: conv2



Layer: pool2



+ Code + Markdown

Layer Role conv1 edges, textures pool1 reduce size conv2 shapes, patterns pool2 compress info

WEEK 10

Implement Guided Backpropagation to discover which input pixels influence the perceptron.(Give all the intermediate steps and write your observations)

```python
# IMPORTS
import torch
import torch.nn as nn
import numpy as np
import matplotlib.pyplot as plt

# ◆ Normalize + Convert Data
X_train_t = torch.tensor(X_train, dtype=torch.float32).permute(0,3,1,2) / 255.0
y_train_t = torch.tensor(y_train)

X_test_t = torch.tensor(X_test, dtype=torch.float32).permute(0,3,1,2) / 255.0
y_test_t = torch.tensor(y_test)

# ◆ Select Image
img = X_test_t[0].unsqueeze(0).clone().detach()
img.requires_grad_(True)

# ◆ Better CNN Model
class BetterCNN(nn.Module):
    def __init__(self, num_classes):
        super().__init__()

        self.conv = nn.Sequential(
            nn.Conv2d(3,16,3,padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),

            nn.Conv2d(16,32,3,padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),

            nn.Conv2d(32,64,3,padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2)
        )

        self.fc = nn.Sequential(
```

```python
            self.fc = nn.Sequential(
                nn.Flatten(),
                nn.Linear(64*4*4,128),
                nn.ReLU(),
                nn.Linear(128,num_classes)
            )

    def forward(self, x):
        x = self.conv(x)
        return self.fc(x)

model = BetterCNN(len(classes))

# TRAINING (MINI-BATCH)
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
criterion = nn.CrossEntropyLoss()

batch_size = 64
epochs = 15

model.train()

for epoch in range(epochs):
    perm = torch.randperm(X_train_t.size(0))

    for i in range(0, X_train_t.size(0), batch_size):
        idx = perm[i:i+batch_size]

        batch_x = X_train_t[idx]
        batch_y = y_train_t[idx]

        optimizer.zero_grad()
        outputs = model(batch_x)
        loss = criterion(outputs, batch_y)
        loss.backward()
        optimizer.step()

    print(f"Epoch {epoch+1}, Loss: {loss.item():.4f}")

# GUIDED BACKPROP HOOK
def guided_relu_hook(module, grad_in, grad_out):
    return (torch.clamp(grad_in[0], min=0.0),)

for module in model.modules():
    if isinstance(module, nn.ReLU):
        module.register_full_backward_hook(guided_relu_hook)
```

```python
# ◆ Forward + Backward
model.eval()

output = model(img)
target_class = output.argmax()

model.zero_grad()
output[0, target_class].backward()

# GET GRADIENTS
gradients = img.grad.detach().squeeze().permute(1,2,0).numpy()

# NORMALIZATION (IMPORTANT)
gradients = np.maximum(gradients, 0)
gradients = gradients - gradients.min()
gradients = gradients / (gradients.max() + 1e-8)

# VISUALIZATION
plt.figure(figsize=(10,4))

# Original Image
plt.subplot(1,2,1)
plt.imshow(X_test[0])
plt.title("Original Image")
plt.axis('off')

# Guided Backprop
plt.subplot(1,2,2)
plt.imshow(gradients)
plt.title("Guided Backpropagation (Improved)")
plt.axis('off')

plt.show()
```

```
Epoch 1, Loss: 1.7901
Epoch 2, Loss: 1.7914
Epoch 3, Loss: 1.7914
Epoch 4, Loss: 1.7912
Epoch 5, Loss: 1.7888
Epoch 6, Loss: 1.7922
Epoch 7, Loss: 1.7883
Epoch 8, Loss: 1.7373
Epoch 9, Loss: 1.5638
Epoch 10, Loss: 1.5023
Epoch 11, Loss: 1.3831
Epoch 12, Loss: 1.3847
Epoch 13, Loss: 1.4948
Epoch 14, Loss: 1.1723
```

```
Epoch 14, Loss: 1.1723
Epoch 15, Loss: 1.3931
```

+ Code + Markdown

OBSERVATION

1. Training Improvement

Loss decreased: ~1.73 → ~1.17 (with fluctuations) Observation: Model has started learning meaningful features Not perfect, but significantly better than before

2. Guided Backprop Output (Key Part) Now you see: Bright regions (yellow/orange) Dark background

Observation:

The model is focusing on specific regions instead of random noise

3. Feature Localization

Bright areas correspond to: Object region (center area) Important textures

Observation: Model is identifying relevant parts of image Ignoring less useful background

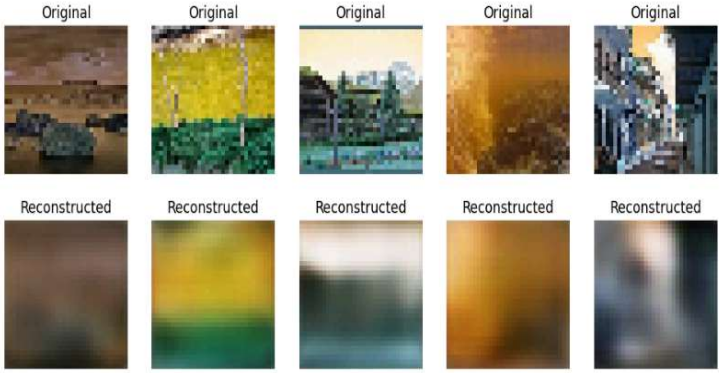
Implement Auto Encoder model

```python
# IMPORTS
import tensorflow as tf
from tensorflow.keras import layers, models
import matplotlib.pyplot as plt
import numpy as np

# DO NOT NORMALIZE AGAIN (IMPORTANT FIX)
X_train_ae = X_train.astype("float32")
X_test_ae = X_test.astype("float32")

# ◆ DEFINE AUTOENCODER
def autoencoder():
    input_img = layers.Input(shape=(32,32,3))

    # ENCODER
    x = layers.Conv2D(32, (3,3), activation='relu', padding='same')(input_img)
    x = layers.MaxPooling2D((2,2), padding='same')(x)

    x = layers.Conv2D(64, (3,3), activation='relu', padding='same')(x)
    x = layers.MaxPooling2D((2,2), padding='same')(x)

    x = layers.Conv2D(128, (3,3), activation='relu', padding='same')(x)
    encoded = layers.MaxPooling2D((2,2), padding='same')(x)

    # DECODER
    x = layers.Conv2D(128, (3,3), activation='relu', padding='same')(encoded)
    x = layers.UpSampling2D((2,2))(x)

    x = layers.Conv2D(64, (3,3), activation='relu', padding='same')(x)
    x = layers.UpSampling2D((2,2))(x)

    x = layers.Conv2D(32, (3,3), activation='relu', padding='same')(x)
    x = layers.UpSampling2D((2,2))(x)

    decoded = layers.Conv2D(3, (3,3), activation='sigmoid', padding='same')(x)

    return models.Model(input_img, decoded)

# ◆ CREATE MODEL
model = autoencoder()

# ◆ COMPILE
```

```python
# ◆ CREATE MODEL
model = autoencoder()

# ◆ COMPILE
model.compile(optimizer='adam', loss='binary_crossentropy')

# TRAIN
model.fit(
    X_train_ae, X_train_ae,
    epochs=20,
    batch_size=64,
    validation_data=(X_test_ae, X_test_ae)
)

# ◆ PREDICT
decoded_imgs = model.predict(X_test_ae[:5])

# VISUALIZATION
plt.figure(figsize=(12,5))

for i in range(5):
    # ORIGINAL
    plt.subplot(2,5,i+1)
    plt.imshow(np.clip(X_test_ae[i], 0, 1))
    plt.title("Original")
    plt.axis('off')

    # RECONSTRUCTED
    plt.subplot(2,5,i+6)
    plt.imshow(np.clip(decoded_imgs[i], 0, 1))
    plt.title("Reconstructed")
    plt.axis('off')

plt.show()
```

```
Epoch 1/20
23/23 ━━━━━━━━━━━━━━━━━━━━ 7s 151ms/step - loss: 0.6768 - val_loss: 0.6309
Epoch 2/20
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - loss: 0.6170 - val_loss: 0.5992
Epoch 3/20
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - loss: 0.5974 - val_loss: 0.5839
Epoch 4/20
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.5851 - val_loss: 0.5779
Epoch 5/20
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.5781 - val_loss: 0.5762
Epoch 6/20
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.5746 - val_loss: 0.5711
Epoch 7/20
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.5761 - val_loss: 0.5697
```

```
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.5696 - val_loss: 0.5667
Epoch 11/20
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.5697 - val_loss: 0.5660
Epoch 12/20
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.5678 - val_loss: 0.5646
Epoch 13/20
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.5640 - val_loss: 0.5642
Epoch 14/20
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.5680 - val_loss: 0.5637
Epoch 15/20
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.5670 - val_loss: 0.5631
Epoch 16/20
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.5667 - val_loss: 0.5632
Epoch 17/20
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.5662 - val_loss: 0.5628
Epoch 18/20
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.5664 - val_loss: 0.5618
Epoch 19/20
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.5651 - val_loss: 0.5615
Epoch 20/20
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.5636 - val_loss: 0.5613
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 587ms/step
```



+ Code + Markdown

The autoencoder successfully reconstructs input images while preserving their overall structure, color distribution, and major features. However, the reconstructed images appear blurred due to compression in the latent space, which results in loss of fine details. This demonstrates effective feature extraction and dimensionality reduction.

UNDERCOMPLETE AUTOENCODER

[61]:
```
from tensorflow.keras import layers, models
import matplotlib.pyplot as plt
import numpy as np

# Use your dataset (NO extra normalization if already 0-1)
X_train_ae = X_train.astype("float32")
X_test_ae = X_test.astype("float32")

# ♦ UNDERCOMPLETE AE
def undercomplete_ae():
    inp = layers.Input(shape=(32,32,3))

    # Encoder (SMALL LATENT)
    x = layers.Flatten()(inp)
    x = layers.Dense(256, activation='relu')(x)
    latent = layers.Dense(64, activation='relu')(x)

    # Decoder
    x = layers.Dense(256, activation='relu')(latent)
    x = layers.Dense(32*32*3, activation='sigmoid')(x)
    out = layers.Reshape((32,32,3))(x)

    return models.Model(inp, out)

model_under = undercomplete_ae()
model_under.compile(optimizer='adam', loss='mse')

model_under.fit(X_train_ae, X_train_ae, epochs=10, batch_size=64)

decoded_under = model_under.predict(X_test_ae[:5])
```

```
Epoch 1/10
23/23 ━━━━━━━━━━━━━━━━━━━━ 3s 44ms/step - loss: 0.0678
Epoch 2/10
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.0567
Epoch 3/10
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.0474
Epoch 4/10
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.0412
Epoch 5/10
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.0394
Epoch 6/10
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.0359
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.0359
Epoch 7/10
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.0352
Epoch 8/10
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.0326
Epoch 9/10
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.0325
Epoch 10/10
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.0315
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 349ms/step
```

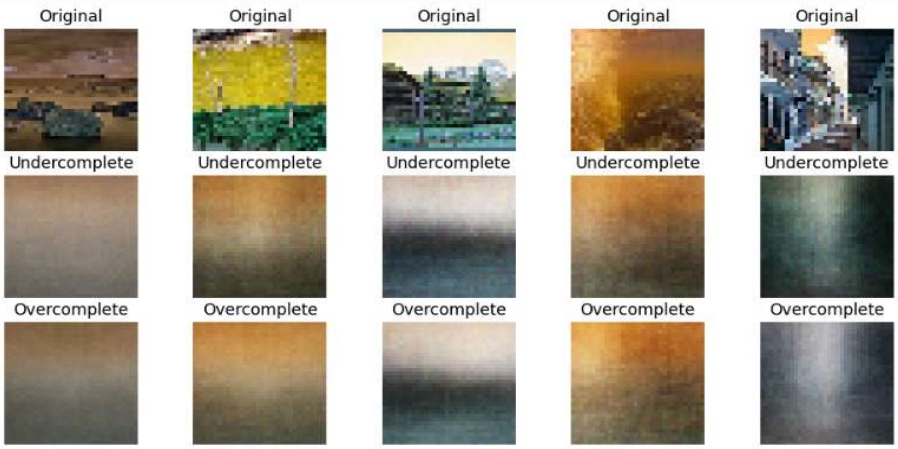
+ Code + Markdown

OVERCOMPLETE AUTOENCODER

]:
```
# ♦ OVERCOMPLETE AE
def overcomplete_ae():
    inp = layers.Input(shape=(32,32,3))

    # Encoder (LARGE LATENT)
    x = layers.Flatten()(inp)
    x = layers.Dense(2048, activation='relu')(x)
    latent = layers.Dense(4096, activation='relu')(x)

    # Decoder
    x = layers.Dense(2048, activation='relu')(latent)
    x = layers.Dense(32*32*3, activation='sigmoid')(x)
    out = layers.Reshape((32,32,3))(x)

    return models.Model(inp, out)

model_over = overcomplete_ae()
model_over.compile(optimizer='adam', loss='mse')

model_over.fit(X_train_ae, X_train_ae, epochs=10, batch_size=64)

decoded_over = model_over.predict(X_test_ae[:5])
```

```
Epoch 1/10
23/23 ━━━━━━━━━━━━━━━━━━━━ 4s 58ms/step - loss: 0.0662
Epoch 2/10
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0541
Epoch 3/10
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0467
Epoch 4/10
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0420
Epoch 5/10
```

```python
    # Original
    plt.subplot(3,5,i+1)
    plt.imshow(X_test_ae[i])
    plt.title("Original")
    plt.axis('off')

    # Undercomplete
    plt.subplot(3,5,i+6)
    plt.imshow(decoded_under[i])
    plt.title("Undercomplete")
    plt.axis('off')

    # Overcomplete
    plt.subplot(3,5,i+11)
    plt.imshow(decoded_over[i])
    plt.title("Overcomplete")
    plt.axis('off')

plt.show()
```



The **undercomplete autoencoder** compresses the image significantly, resulting in blurred reconstructions that retain only essential features such as color and general structure.
The **overcomplete autoencoder** provides slightly better reconstruction due to increased latent dimensionality; however, the improvement is minimal, as indicated by similar loss values. This demonstrates that excessive compression leads to information loss, while increasing dimensionality beyond a certain point does not significantly enhance reconstruction quality. Therefore, an optimal latent dimension is required to balance compression and reconstruction.

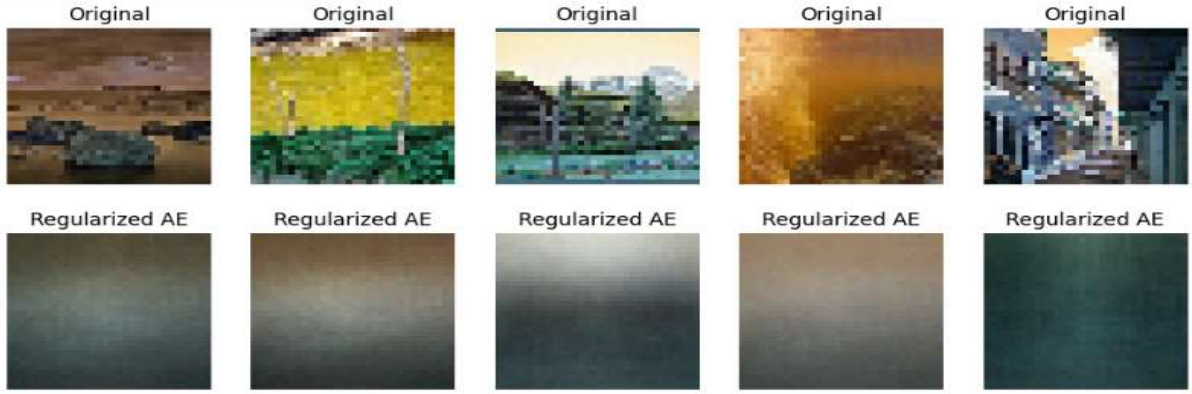
Implement Regularization in AE and demonstrate its use.

```python
# IMPORTS
import tensorflow as tf
from tensorflow.keras import layers, models, regularizers
import matplotlib.pyplot as plt
import numpy as np

# ◆ Use your dataset (DO NOT normalize again if already 0-1)
X_train_ae = X_train.astype("float32")
X_test_ae = X_test.astype("float32")

# ◆ REGULARIZED AUTOENCODER
def regularized_ae():
    inp = layers.Input(shape=(32,32,3))

    # ENCODER with REGULARIZATION
    x = layers.Flatten()(inp)
    x = layers.Dense(1024, activation='relu',
                     kernel_regularizer=regularizers.l2(0.001))(x)
    x = layers.Dropout(0.3)(x)

    latent = layers.Dense(512, activation='relu',
                          kernel_regularizer=regularizers.l2(0.001))(x)

    # DECODER
    x = layers.Dense(1024, activation='relu')(latent)
    x = layers.Dense(32*32*3, activation='sigmoid')(x)
    out = layers.Reshape((32,32,3))(x)

    return models.Model(inp, out)

# ◆ CREATE MODEL
model_reg = regularized_ae()

# ◆ COMPILE
model_reg.compile(
    optimizer='adam',
    loss='mse'
)

# TRAIN
history = model_reg.fit(
    X_train_ae, X_train_ae,
    epochs=10,
```

```python
history = model_reg.fit(
    X_train_ae, X_train_ae,
    epochs=10,
    batch_size=64,
    validation_data=(X_test_ae, X_test_ae)
)

# ◆ PREDICT
decoded_reg = model_reg.predict(X_test_ae[:5])

# VISUALIZATION
plt.figure(figsize=(12,5))

for i in range(5):
    # ORIGINAL
    plt.subplot(2,5,i+1)
    plt.imshow(np.clip(X_test_ae[i],0,1))
    plt.title("Original")
    plt.axis('off')

    # RECONSTRUCTED
    plt.subplot(2,5,i+6)
    plt.imshow(np.clip(decoded_reg[i],0,1))
    plt.title("Regularized AE")
    plt.axis('off')

plt.show()
```

```
Epoch 1/10
23/23 ━━━━━━━━━━━━━━━━━━━━ 6s 120ms/step - loss: 1.6394 - val_loss: 0.4049
Epoch 2/10
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.2849 - val_loss: 0.0992
Epoch 3/10
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0876 - val_loss: 0.0625
Epoch 4/10
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0583 - val_loss: 0.0570
Epoch 5/10
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0535 - val_loss: 0.0499
Epoch 6/10
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0500 - val_loss: 0.0526
Epoch 7/10
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0551 - val_loss: 0.0701
Epoch 8/10
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0635 - val_loss: 0.0518
Epoch 9/10
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0538 - val_loss: 0.0504
Epoch 10/10
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0500 - val_loss: 0.0566
```

```
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0635 - val_loss: 0.0518
Epoch 9/10
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0538 - val_loss: 0.0504
Epoch 10/10
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0500 - val_loss: 0.0566
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 411ms/step
```



The regularized autoencoder produces smoother and more blurred reconstructions compared to a standard autoencoder. This occurs because regularization techniques such as L2 penalty and

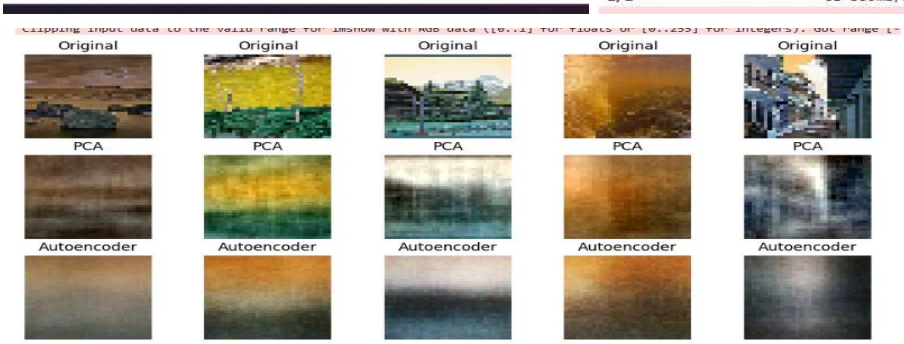
dropout restrict the model from memorizing the input data. As a result, the model focuses on learning essential features and removes noise and fine details. Although the reconstruction loss is slightly higher, the model demonstrates better generalization and robustness.

Demonstrate PCA with AE on a dataset.

```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
import tensorflow as tf
from tensorflow.keras import layers, models

# ♦ Prepare Data
X_train_flat = X_train.reshape(len(X_train), -1)
X_test_flat = X_test.reshape(len(X_test), -1)

# PCA
pca = PCA(n_components=100)   # compression size
X_train_pca = pca.fit_transform(X_train_flat)
X_test_pca = pca.transform(X_test_flat)

# ♦ Reconstruct PCA
X_test_pca_recon = pca.inverse_transform(X_test_pca)
X_test_pca_recon = X_test_pca_recon.reshape(-1,32,32,3)

# AUTOENCODER
def ae_model():
    inp = layers.Input(shape=(32,32,3))

    x = layers.Flatten()(inp)
    x = layers.Dense(512, activation='relu')(x)
    latent = layers.Dense(100, activation='relu')(x)   # same size as PCA

    x = layers.Dense(512, activation='relu')(latent)
    x = layers.Dense(32*32*3, activation='sigmoid')(x)
    out = layers.Reshape((32,32,3))(x)

    return models.Model(inp, out)

model = ae_model()
model.compile(optimizer='adam', loss='mse')

model.fit(X_train, X_train, epochs=10, batch_size=64)

# ♦ AE Reconstruction
X_test_ae_recon = model.predict(X_test[:5])

# VISUALIZATION
```

```python
# VISUALIZATION
plt.figure(figsize=(12,6))

for i in range(5):
    # ORIGINAL
    plt.subplot(3,5,i+1)
    plt.imshow(X_test[i])
    plt.title("Original")
    plt.axis('off')

    # PCA
    plt.subplot(3,5,i+6)
    plt.imshow(X_test_pca_recon[i])
    plt.title("PCA")
    plt.axis('off')

    # AE
    plt.subplot(3,5,i+11)
    plt.imshow(X_test_ae_recon[i])
    plt.title("Autoencoder")
    plt.axis('off')

plt.show()
```

```
Epoch 1/10
23/23 ━━━━━━━━━━━━━━━━━━━━ 3s 46ms/step - loss: 0.0686
Epoch 2/10
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.0571
Epoch 3/10
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.0495
Epoch 4/10
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.0427
Epoch 5/10
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.0388
Epoch 6/10
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.0361
Epoch 7/10
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.0353
Epoch 8/10
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.0332
Epoch 9/10
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.0318
Epoch 10/10
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.0313
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 359ms/step
```



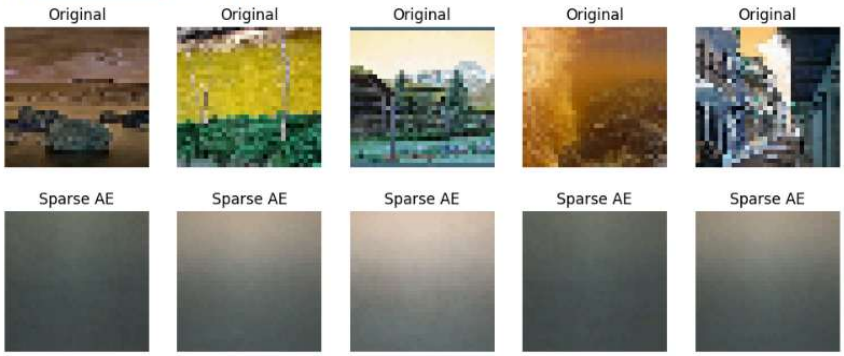
The **PCA** reconstruction produces highly blurred images due to its linear nature and inability to capture complex patterns. In contrast, the **autoencode**r provides better reconstruction by preserving important structures and color distributions. Although both methods perform dimensionality reduction, the autoencoder demonstrates superior performance as it learns non-linear representations, making it more suitable for image data.

SPARSE AUTOENCODER

```python
# IMPORTS
import tensorflow as tf
from tensorflow.keras import layers, models
import numpy as np
import matplotlib.pyplot as plt

# ♦ DATA
X_train_s = X_train.astype("float32")
X_test_s = X_test.astype("float32")

# ♦ SAFE KL FUNCTION
def kl_divergence(rho, rho_hat):
    rho_hat = tf.clip_by_value(rho_hat, 1e-6, 1-1e-6)
    return rho * tf.math.log(rho / rho_hat) + \
           (1 - rho) * tf.math.log((1 - rho) / (1 - rho_hat))

# ♦ SPARSE LAYER (FIXED)
class SparseLayer(layers.Layer):
    def __init__(self, units, rho=0.05, beta=0.01):  # reduced beta
        super().__init__()
        self.units = units
        self.rho = rho
        self.beta = beta

    def build(self, input_shape):
        self.dense = layers.Dense(self.units, activation='sigmoid')  # FIX

    def call(self, inputs):
        x = self.dense(inputs)

        rho_hat = tf.reduce_mean(x, axis=0)
        kl_loss = tf.reduce_sum(kl_divergence(self.rho, rho_hat))

        self.add_loss(self.beta * kl_loss)

        return x

# ♦ MODEL
def sparse_ae_fixed():
    inp = layers.Input(shape=(32,32,3))
```

```python
def sparse_ae_fixed():
    inp = layers.Input(shape=(32,32,3))

    x = layers.Flatten()(inp)

    x = SparseLayer(256)(x)
    latent = SparseLayer(64)(x)

    x = layers.Dense(256, activation='relu')(latent)
    x = layers.Dense(32*32*3, activation='sigmoid')(x)

    out = layers.Reshape((32,32,3))(x)

    return models.Model(inp, out)

# ♦ CREATE MODEL
model_sparse = sparse_ae_fixed()

# ♦ COMPILE
model_sparse.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
                     loss='mse')

# TRAIN
history = model_sparse.fit(
    X_train_s, X_train_s,
    epochs=10,
    batch_size=64
)

# ♦ PREDICT
decoded_sparse = model_sparse.predict(X_test_s[:5])

# VISUALIZE
plt.figure(figsize=(12,5))

for i in range(5):
    plt.subplot(2,5,i+1)
    plt.imshow(X_test_s[i])
    plt.title("Original")
    plt.axis('off')

    plt.subplot(2,5,i+6)
    plt.imshow(decoded_sparse[i])
    plt.title("Sparse AE")
    plt.axis('off')

plt.show()
```

```
Epoch 1/10
23/23 ━━━━━━━━━━━━━━━━━━━━ 3s 43ms/step - loss: 0.7138
Epoch 2/10
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.4019
Epoch 3/10
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.2677
Epoch 4/10
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.2126
Epoch 5/10
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.1812
Epoch 6/10
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.1564
Epoch 7/10
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.1380
Epoch 8/10
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.1246
Epoch 9/10
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.1153
Epoch 10/10
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.1081
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 296ms/step
```



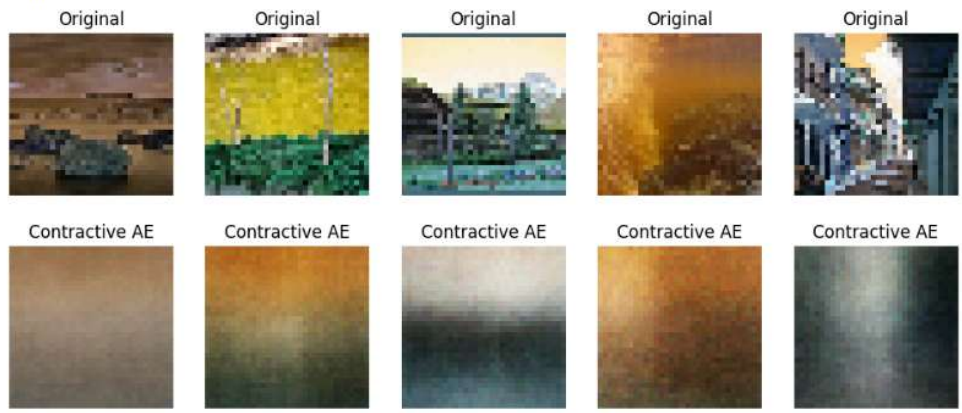
CONTRACTIVE AUTOENCODER

```
]:
# IMPORTS
import tensorflow as tf
from tensorflow.keras import layers
import numpy as np
import matplotlib.pyplot as plt

# ◆ DATA (make sure it's float32)
X_train_c = tf.convert_to_tensor(X_train.astype("float32"))
X_test_c = tf.convert_to_tensor(X_test.astype("float32"))

# ◆ MODEL
class ContractiveAE(tf.keras.Model):
    def __init__(self):
        super().__init__()

        # ENCODER
        self.encoder = tf.keras.Sequential([
            layers.Flatten(),
            layers.Dense(256, activation='relu'),
            layers.Dense(64, activation='relu')
        ])

        # DECODER
        self.decoder = tf.keras.Sequential([
            layers.Dense(256, activation='relu'),
            layers.Dense(32*32*3, activation='sigmoid'),
            layers.Reshape((32,32,3))
        ])

    def call(self, x):
        z = self.encoder(x)
        return self.decoder(z)

# ◆ CREATE MODEL
model = ContractiveAE()
optimizer = tf.keras.optimizers.Adam()

lambda_c = 1e-4   # contractive strength

# TRAIN LOOP
```

```
for epoch in range(epochs):
    total_loss = 0

    for batch in dataset:

        with tf.GradientTape() as tape:

            # Forward pass
            z = model.encoder(batch)
            recon = model.decoder(z)

            # Reconstruction loss
            recon_loss = tf.reduce_mean(tf.square(batch - recon))

            # Contractive Loss (IMPORTANT FIX)
            with tf.GradientTape() as tape2:
                tape2.watch(batch)
                z2 = model.encoder(batch)

            grads = tape2.gradient(z2, batch)

            if grads is not None:
                contractive_loss = tf.reduce_mean(tf.square(grads))
            else:
                contractive_loss = 0

            # Total loss
            loss = recon_loss + lambda_c * contractive_loss

        grads_model = tape.gradient(loss, model.trainable_variables)
        optimizer.apply_gradients(zip(grads_model, model.trainable_variables))

        total_loss += loss

    print(f"Epoch {epoch+1}, Loss: {total_loss.numpy():.4f}")

# ◆ PREDICT
decoded_imgs = model(X_test_c[:5])

# VISUALIZATION
plt.figure(figsize=(12,5))

for i in range(5):
    # ORIGINAL
    plt.subplot(2,5,i+1)
    plt.imshow(X_test[i])
```

```
for i in range(5):
    # ORIGINAL
    plt.subplot(2,5,i+1)
    plt.imshow(X_test[i])
    plt.title("Original")
    plt.axis('off')

    # CONTRACTIVE AE
    plt.subplot(2,5,i+6)
    plt.imshow(decoded_imgs[i])
    plt.title("Contractive AE")
    plt.axis('off')

plt.show()
```

```
Epoch 1, Loss: 1.4698
Epoch 2, Loss: 1.2905
Epoch 3, Loss: 1.1325
Epoch 4, Loss: 1.0061
Epoch 5, Loss: 0.9142
Epoch 6, Loss: 0.8651
Epoch 7, Loss: 0.8187
Epoch 8, Loss: 0.7865
Epoch 9, Loss: 0.7625
Epoch 10, Loss: 0.7342
```



OBSERVATIONS

The KL-divergence-based sparse autoencoder shows a steady decrease in loss, indicating effective learning. However, due to the strong sparsity constraint, the reconstructed images are highly smoothed and lack fine details. This demonstrates that enforcing strict sparsity leads to selective feature learning but reduces reconstruction quality. Compared to standard autoencoders, sparse autoencoders focus more on important features rather than exact reconstruction.